МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

Лабораторна робота №1

з курсу: «Технології побудови розподілених баз даних та знань»

Виконав:
студент групи КН-51мп
Сидько Н.С.

КИЇВ
2026

ЛР1. High Availability в Redis

Завдання:
1. Встановіть API будь-якої мови програмування.
2. Програмно протягом 60 с ймовірнісно дозапишіть 20 нових та оновіть 5 існуючих екземплярів. Зчитайте обсяг байт окремих файлів (aof, rdb)
3. Програмно обчисліть загальну тривалість делегування від вимикання існуючого керівного (основного) і встановлення нового основного.

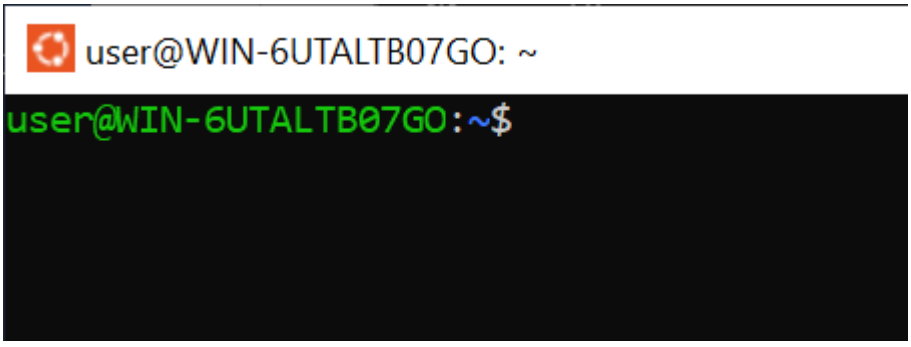
Хід роботи

1) Підготовка серидовища WSL

На початку виконання лабораторної роботи було підготовлено робоче середовище з використанням підсистеми Windows Subsystem for Linux (WSL). Для цього у середовищі Windows було виконано встановлення WSL з дистрибутивом Ubuntu за допомогою команди wsl --install. Після завершення інсталяції було створено користувацький обліковий запис у середовищі Linux.

```
PS C:\Users\user> wsl --install
Скачивание: Ubuntu
Установка: Ubuntu
Дистрибутив успешно установлен. Его можно запустить с помощью "wsl.exe -d Ubuntu"
Запуск Ubuntu...
Provisioning the new WSL instance Ubuntu
This might take a while...
Create a default Unix user account: user
New password:
Retype new password:
passwd: password updated successfully
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
```

Далі було відкрито термінал Ubuntu, у якому виконувались усі подальші дії лабораторної роботи. Використання WSL дозволило працювати з Linux-утилітами та сервером Redis у середовищі, наближеному до реального серверного оточення, що є необхідним для дослідження механізмів реплікації та відмовостійкості.

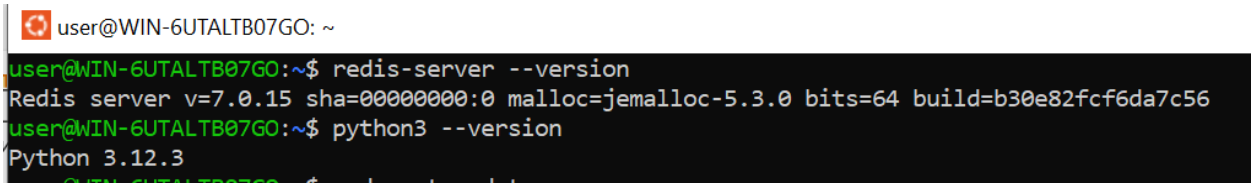


2) Встановлення та перевірка Redis Server і Python

Після підготовки середовища WSL було виконано встановлення необхідного програмного забезпечення для виконання лабораторної роботи. Зокрема, було встановлено Redis Server — систему керування базами даних типу key-value, а також мову програмування Python, яка використовувалась для автоматизації тестування.

Встановлення Redis було виконано за допомогою пакетного менеджера apt, після чого було перевірено його працездатність та версію за допомогою команди redis-server --version. Аналогічно було перевірено встановлення Python за допомогою команди python3 --version.

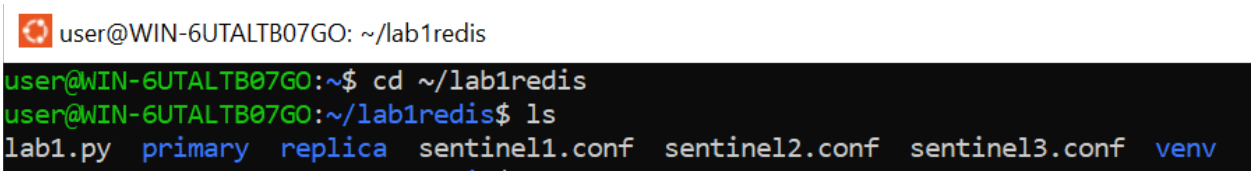
У результаті було встановлено Redis Server версії 7.0.15 та Python версії 3.12.3, що забезпечило можливість подальшого налаштування реплікації, моніторингу та виконання програмного навантаження на систему.



3) Створення структури проекту

Після встановлення необхідного програмного забезпечення було створено структуру директорій для організації компонентів системи Redis. Зокрема, було створено окрему директорію для лабораторної роботи, у якій розміщено конфігураційні файли основного вузла (primary), репліки (replica) та допоміжні файли.

У середовищі Ubuntu (WSL) було створено каталог lab1redis, у якому надалі виконувались усі дії. Усередині нього було створено підкаталоги primary та replica, які використовуються для запуску відповідних екземплярів Redis із власними конфігураціями та файлами збереження даних.

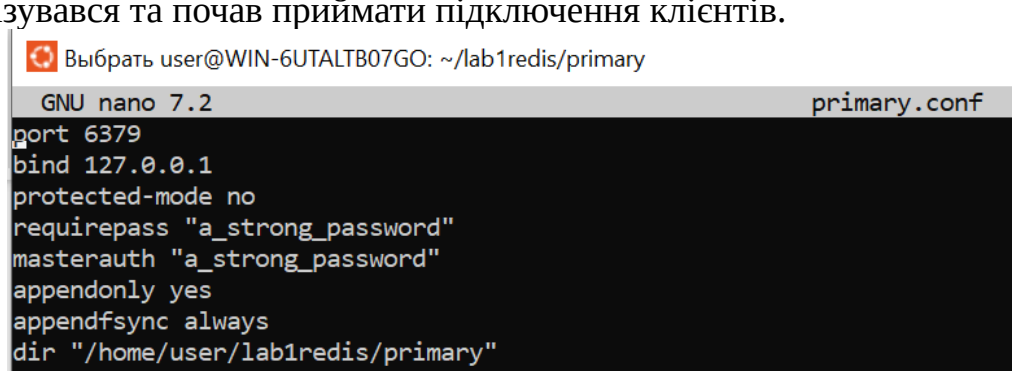


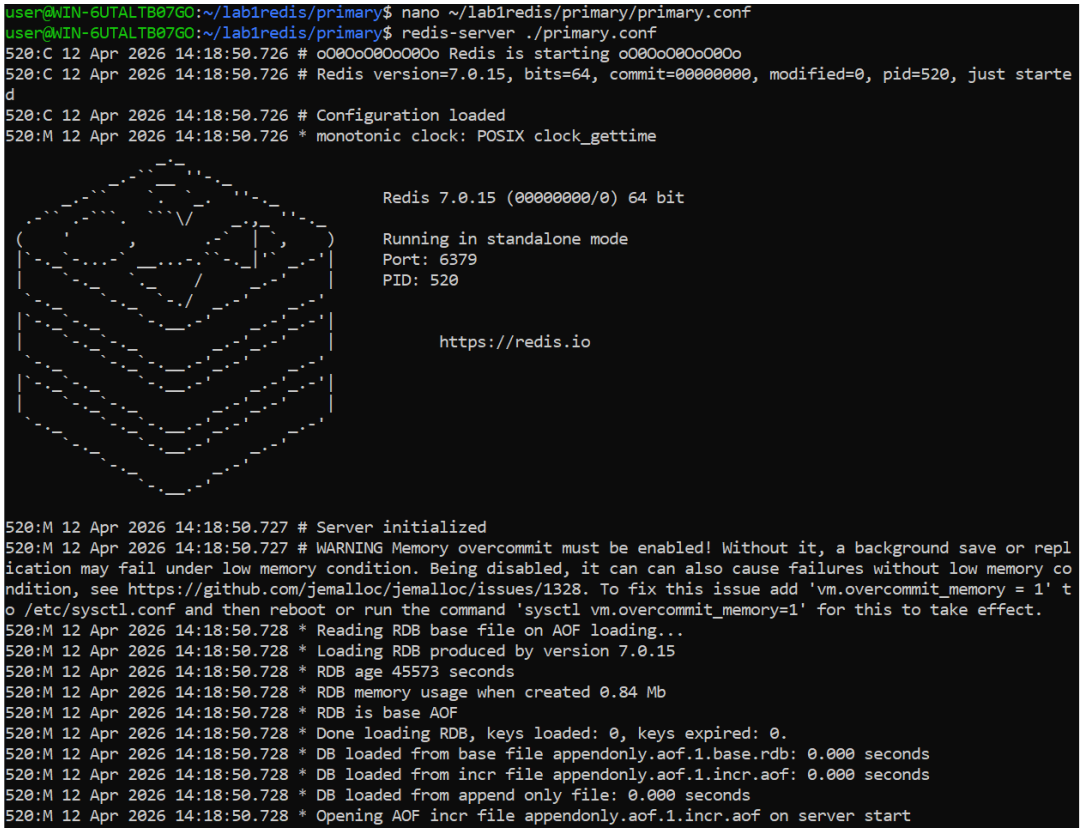
4) Налаштування та запуск основного вузла (Primary)

На даному етапі було виконано налаштування та запуск основного вузла (primary) системи Redis. Для цього у каталозі primary було створено конфігураційний файл primary.conf, у якому задано основні параметри роботи сервера.

Зокрема, було вказано порт підключення, адресу прив’язки, вимкнено захищений режим, а також встановлено пароль доступу. Додатково було увімкнено механізм журналювання AOF (Append Only File), що забезпечує збереження всіх змін у базі даних.

Після створення конфігураційного файлу було виконано запуск Redis-сервера з використанням цього файлу. У результаті сервер успішно ініціалізувався та почав приймати підключення клієнтів.





переконаймось, що primary реально живий:

```
user@WIN-6UTALTB07GO:~/lab1redis/primary$ redis-cli -p 6379 -a a_strong_password --no-auth-warning PING
PONG
user@WIN-6UTALTB07GO:~/lab1redis/primary$ redis-cli -p 6379 -a a_strong_password --no-auth-warning INFO replication | head -n 5
# Replication
role:master
connected_slaves:0
master_failover_state:no-failover
master_replid:441dc681e611a632c280cb6a89e31da7e40ab1e2
user@WIN-6UTALTB07GO:~/lab1redis/primary$
```

5) Налаштування та запуск репліки (Replica)

Для забезпечення резервного дублювання даних основного вузла було створено другий екземпляр Redis, який виконуватиме роль replica — дублювального сервера, що у реальному часі отримує всі операції запису з primary і зберігає копію бази даних. У каталозі ~/lab1redis/replica за допомогою редактора nano було створено конфігураційний файл replica.conf.

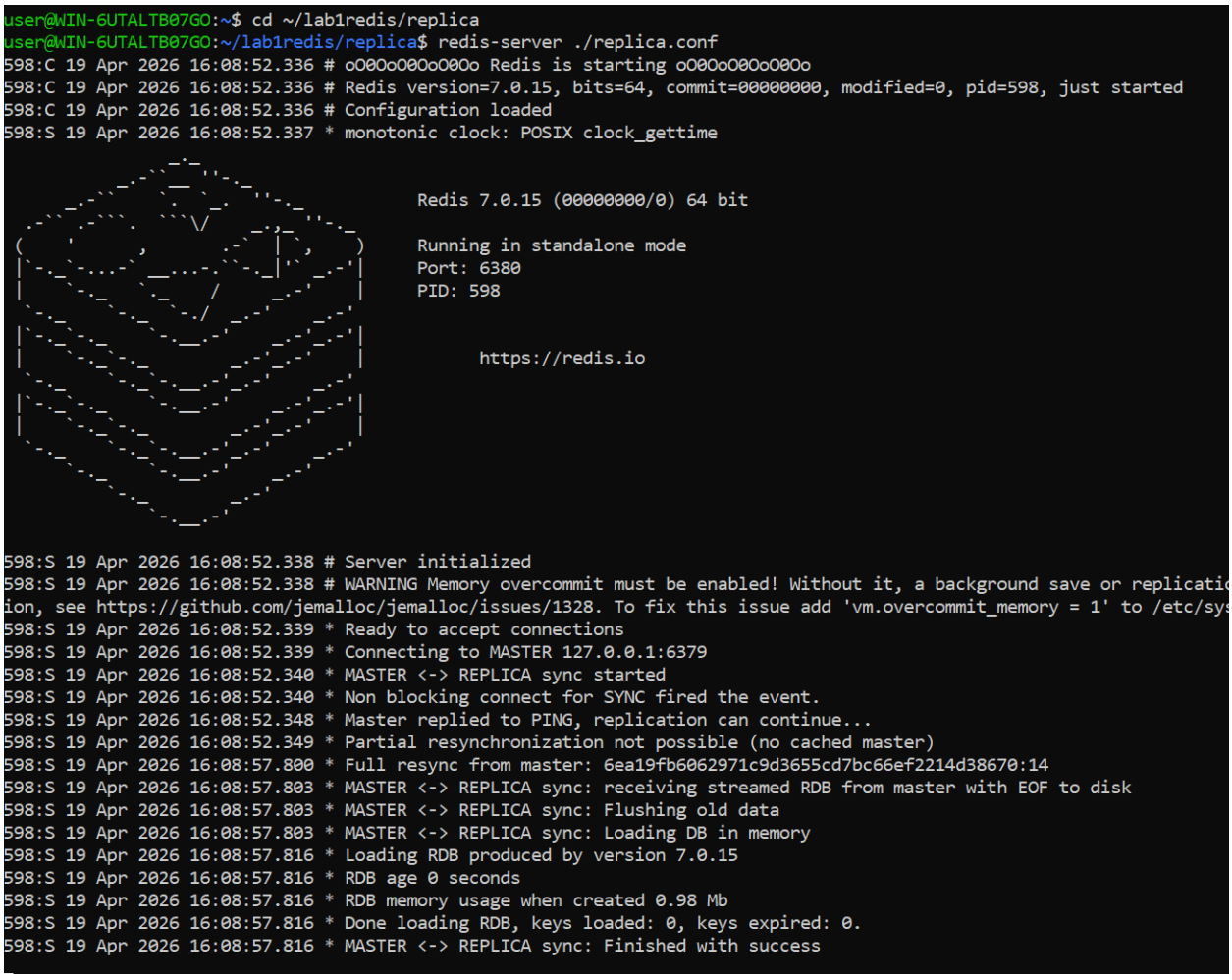
Директивою port 6380 задано окремий порт для дублювального сервера, оскільки на одній машині не можуть одночасно працювати два екземпляри Redis на одному порту. Ключова директива replicaof 127.0.0.1 6379 вказує, що цей сервер є реплікою основного вузла за адресою 127.0.0.1 на порту 6379. Пароль masterauth a_strong_password необхідний для автентифікації дублювального сервера на основному, оскільки primary налаштований з директивою requirepass; без нього реплікація не встановиться. Власна директива requirepass a_strong_password задає пароль для клієнтів, що підключаються безпосередньо до replica.

Каталог зберігання даних задано директивою dir, а ім'я файлу RDB — параметром dbfilename my_backup_file.rdb відповідно до умови завдання. Директива save 30 5 встановлює правило створення RDB-знімка: якщо протягом 30 секунд змінилось 5 і більше ключів, сервер автоматично формує знімок бази у файл my_backup_file.rdb. Таким чином replica поєднує безперервну потокову реплікацію з періодичним збереженням знімків на диск.

```
user@WIN-6UTALTB07GO: ~/lab1redis/replica
  GNU nano 7.2                                  replica.conf *
# Порт дублювального сервера
port 6380
bind 127.0.0.1
protected-mode no
# Адреса та порт основного екземпляра
replicaof 127.0.0.1 6379
# Код автентифікації основного екземпляра
masterauth a_strong_password
# Код автентифікації щодо дублювального екземпляра
requirepass a_strong_password
# Каталог зберігання даних replica
dir /home/user/lab1redis/replica
# Найменування файлу зберігання в зовнішньому носії даних
dbfilename my_backup_file.rdb
# Дублювання, якщо 5 ключів оновлено протягом 30 с
save 30 5
```

Дублювальний сервер було запущено у власному терміналі командою redis-server ./replica.conf у каталозі ~/lab1redis/replica. У журналі запуску replica було зафіксовано повну послідовність встановлення зв'язку з основним вузлом: підключення до MASTER 127.0.0.1:6379, успішне відповідання primary на PING (Master replied to PING, replication can continue), запит на повну синхронізацію (Full resync from master), отримання RDB-знімка бази (MASTER <-> REPLICA sync: receiving bytes from master to disk), завантаження його в оперативну пам'ять (Loading DB in memory) та завершальне повідомлення MASTER <-> REPLICA sync: Finished with success. Завершальний рядок Ready to accept connections tcp підтвердив готовність replica до обслуговування запитів на порту 6380.

Одночасно у журналі primary було зафіксовано зустрічні записи: Replica 127.0.0.1:6380 asks for synchronization, Full resync requested by replica, Starting BGSAVE for SYNC with target: replicas sockets та Synchronization with replica 127.0.0.1:6380 succeeded. Це підтвердило, що primary коректно виконав формування знімка бази у фоновому режимі та передав його дублювальному серверу. Таким чином було завершено первинну повну синхронізацію (full resync), після якої replica перейшла у режим потокової реплікації — отримання команд запису з primary у реальному часі через replication backlog.



```
user@WIN-6UTALTB07GO:~/lab1redis/replica$ redis-cli -p 6380 -a a_strong_password --no-auth-warning PING
PONG
user@WIN-6UTALTB07GO:~/lab1redis/replica$ redis-cli -p 6380 -a a_strong_password --no-auth-warning INFO replicati
on | head -n 10
# Replication
role:slave
master_host:127.0.0.1
master_port:6379
master_link_status:up
master_last_io_seconds_ago:1
master_sync_in_progress:0
slave_read_repl_offset:98
slave_repl_offset:98
slave_priority:100
```

6) Перевірка доступності primary і replica

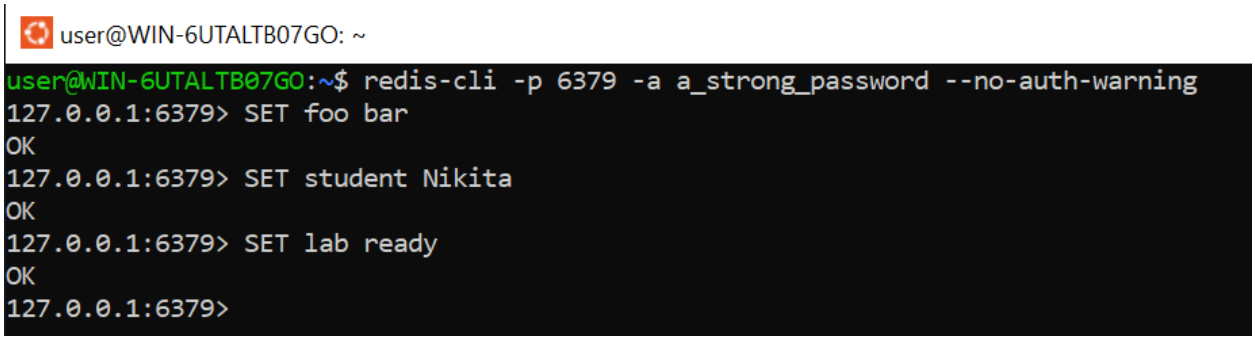
Після запуску обох екземплярів було виконано базову перевірку їхньої доступності за допомогою команди PING з автентифікацією паролем. Для основного сервера на порту 6379 і для дублювального на порту 6380 було отримано відповідь PONG, що підтвердило коректний запуск обох екземплярів Redis та їхню готовність обслуговувати клієнтські запити. Використання ключа -a a_strong_password необхідне, оскільки обидва сервери налаштовані з директивою requirepass, і будь-яка команда без автентифікації повертатиме помилку NOAUTH Authentication required.

```
user@WIN-6UTALTB07GO:~/lab1redis/replica$ redis-cli -p 6379 -a a_strong_password --no-auth-warning PING
PONG
user@WIN-6UTALTB07GO:~/lab1redis/replica$ redis-cli -p 6380 -a a_strong_password --no-auth-warning PING
PONG
user@WIN-6UTALTB07GO:~/lab1redis/replica$
```

7) Перевірка реплікації командою MONITOR

На replica запускаємо режим MONITOR, який друкує кожну команду, що прилітає на цей сервер. Паралельно на primary пишемо ключі. Якщо реплікація працює — команди побачимо у вікні MONITOR.

```
user@WIN-6UTALTB07GO:~/lab1redis/replica$ redis-cli -p 6380 -a a_strong_password --no-auth-warning
127.0.0.1:6380> MONITOR
OK
1776615397.486823 [0 127.0.0.1:6379] "ping"
1776615408.185793 [0 127.0.0.1:6379] "ping"
```





Для перевірки передачі команд від основного вузла до дублювального було використано механізм MONITOR, який у реальному часі виводить у консоль усі запити, що обслуговує Redis-сервер. У терміналі допоміжної вкладки було встановлено з'єднання з replica командою redis-cli -p 6380 -a a_strong_password, після чого виконано команду MONITOR, яка перевела сервер у режим потокового виводу запитів.

Паралельно у окремому терміналі було встановлено з'єднання з primary (redis-cli -p 6379 -a a_strong_password) і послідовно виконано три операції запису: SET foo bar, SET student Nikita та SET lab ready. На кожну з них основний сервер повернув відповідь OK. У цей же момент у вікні MONITOR на replica з'явились відповідні записи із часовою міткою та IP-адресою клієнта, який ініціював команду на primary, зокрема "set" "foo" "bar", "set" "student" "Nikita" та "set" "lab" "ready". Це підтвердило, що дублювальний сервер дійсно отримує повний потік команд запису через механізм потокової реплікації Redis: кожна команда, виконана на primary, у режимі реального часу передається через replication backlog на replica та застосовується до її копії бази.

8) Перевірка файлів персистентності

Для підвищення надійності збереження даних режим журналювання AOF було змінено з встановленого раніше everysec на найбільш суворий — always, який виконує примусовий скид журналу на диск після кожної команди запису, забезпечуючи максимальну властивість Durability. Оскільки змінювати параметри потрібно було без зупинки основного сервера, було використано механізм динамічного налаштування Redis. У клієнті redis-cli, підключеному до primary, командою CONFIG GET appendfsync було підтверджено поточне значення everysec, після чого командами CONFIG SET appendonly yes та CONFIG SET appendfsync always у оперативному режимі було змінено параметри сервера. Повторне виконання CONFIG GET appendfsync повернуло нове значення always, що підтвердило успішну зміну. Щоб нові параметри збереглися у конфігураційному файлі на випадок перезапуску сервера, було виконано команду CONFIG REWRITE, яка повернула OK і фізично переписала файл primary.conf з новими значеннями директив.

```
user@WIN-6UTALTB07GO:~/lab1redis/replica$ redis-cli -p 6379 -a a_strong_password --no-auth-warning
127.0.0.1:6379> CONFIG GET appendfsync
1) "appendfsync"
2) "everysec"
127.0.0.1:6379> CONFIG SET appendonly yes
OK
127.0.0.1:6379> CONFIG SET appendfsync always
OK
127.0.0.1:6379> CONFIG GET appendfsync
1) "appendfsync"
2) "always"
```

Додатково, для підготовки подальшої перевірки механізму персистентності, у базу даних було записано два тестові ключі: persist:test1 зі значенням hello та persist:test2 зі значенням world. Команди GET persist:test1 і GET persist:test2 повернули відповідно "hello" та "world", що підтвердило успішний запис ключів у базу. Ці ключі надалі використовуватимуться для перевірки відновлення даних після перезапуску primary.

```
127.0.0.1:6379> CONFIG REWRITE
OK
127.0.0.1:6379> SET persist:test1 hello
OK
127.0.0.1:6379> SET persist:test2 world
OK
127.0.0.1:6379> GET persist:test1
"hello"
127.0.0.1:6379> GET persist:test2
"world"
127.0.0.1:6379> EXIT
```

Після виконання операцій запису було перевірено структуру файлів, створених механізмами персистентності Redis. У каталозі основного сервера ~/lab1redis/primary було виявлено конфігураційний файл primary.conf та підкаталог appendonlydir, у якому, у відповідності до формату Redis 7, зберігаються файли AOF: базовий знімок primary.aof.1.base.rdb, що містить стан бази на момент останнього перезапису журналу, інкрементний файл primary.aof.1.incr.aof, у який послідовно дозаписуються всі нові команди, та маніфест primary.aof.manifest, що зв'язує ці файли у логічне ціле. Саме у файл primary.aof.1.incr.aof у режимі appendfsync always потрапили команди SET persist:test1 hello та SET persist:test2 world.

У каталозі дублювального сервера ~/lab1redis/replica, крім конфігураційного файлу replica.conf, було виявлено файл my_backup_file.rdb, сформований автоматично відповідно до правила save 30 5: після запису п'яти і більше ключів протягом 30 секунд replica створила бінарний знімок бази. Таким чином система одночасно використовує два незалежні механізми надійного збереження даних — журналювання AOF на стороні primary та періодичні RDB-знімки на стороні replica.

```
user@WIN-6UTALTB07GO:~/lab1redis/replica$ ls -la ~/lab1redis/primary
total 20
drwxr-xr-x 3 user user 4096 Apr 19 16:03 .
drwxr-xr-x 6 user user 4096 Apr 18 21:43 ..
drwxr-xr-x 2 user user 4096 Apr 18 21:44 appendonlydir
-rw-r--r-- 1 user user   89 Apr 18 21:49 dump.rdb
-rw-r--r-- 1 user user  574 Apr 18 21:44 primary.conf
user@WIN-6UTALTB07GO:~/lab1redis/replica$ ls -la ~/lab1redis/primary/appendonlydir
total 20
drwxr-xr-x 2 user user 4096 Apr 18 21:44 .
drwxr-xr-x 3 user user 4096 Apr 19 16:03 ..
-rw-r--r-- 1 user user   89 Apr 18 21:44 primary.aof.1.base.rdb
-rw-r--r-- 1 user user  125 Apr 19 16:17 primary.aof.1.incr.aof
-rw-r--r-- 1 user user   82 Apr 18 21:44 primary.aof.manifest
user@WIN-6UTALTB07GO:~/lab1redis/replica$ ls -la ~/lab1redis/replica
total 16
drwxr-xr-x 2 user user 4096 Apr 19 16:08 .
drwxr-xr-x 6 user user 4096 Apr 18 21:43 ..
-rw-r--r-- 1 user user  172 Apr 19 16:08 my_backup_file.rdb
-rw-r--r-- 1 user user  747 Apr 19 16:07 replica.conf
```

9) Перезапуск primary та відновлення даних

Для перевірки того, що налаштований механізм AOF справді забезпечує надійне відновлення даних після аварійного вимикання сервера, основний вузол було примусово зупинено. У вікні, де працював primary, було надіслано сигнал SIGINT комбінацією Ctrl+C, після чого Redis у журналі зафіксував виконання fsync() над файлом AOF (Calling fsync() on the AOF file), коректне видалення pid-файлу та штатне завершення процесу (Redis is now ready to exit, bye bye). Після цього команда redis-cli -p 6379 ... PING повертала помилку Could not connect to Redis at 127.0.0.1:6379: Connection refused, що підтверджувало повну зупинку основного сервера. Одночасно у журналі replica з'явились повідомлення Connection with master lost та періодичні спроби повторного підключення — replica автоматично намагалась відновити реплікаційний канал.

```
^C586:signal-handler (1776618247) Received SIGINT scheduling shutdown...
586:M 19 Apr 2026 17:04:07.662 # User requested shutdown...
586:M 19 Apr 2026 17:04:07.662 * 1 of 1 replicas are in sync when shutting down.
586:M 19 Apr 2026 17:04:07.662 * Calling fsync() on the AOF file.
586:M 19 Apr 2026 17:04:07.663 * Saving the final RDB snapshot before exiting.
586:M 19 Apr 2026 17:04:07.682 * DB saved on disk
586:M 19 Apr 2026 17:04:07.682 # Redis is now ready to exit, bye bye...
user@WIN-6UTALTB07GO:~/lab1redis/primary$
```

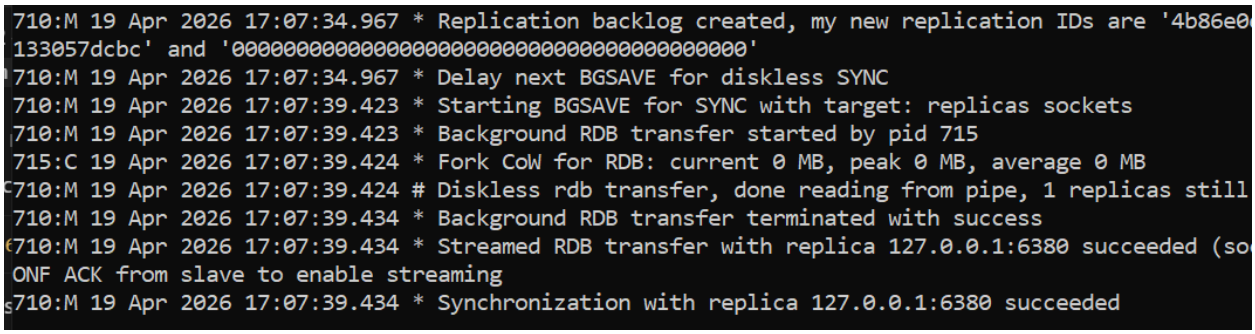
```
user@WIN-6UTALTB07GO:~/lab1redis/replica$ redis-cli -p 6379 -a a_strong_password --no-auth-warning PING
Could not connect to Redis at 127.0.0.1:6379: Connection refused
```

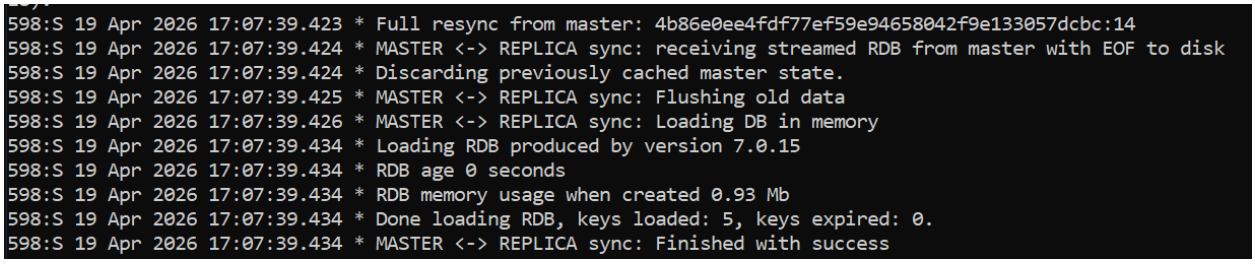
Выбрать user@WIN-6UTALTB07GO: ~/lab1redis/replica

```
598:S 19 Apr 2026 16:08:57.816 * RDB age 0 seconds
598:S 19 Apr 2026 16:08:57.816 * RDB memory usage when created 0.98 Mb
598:S 19 Apr 2026 16:08:57.816 * Done loading RDB, keys loaded: 0, keys expired: 0.
598:S 19 Apr 2026 16:08:57.816 * MASTER <-> REPLICA sync: Finished with success
598:S 19 Apr 2026 16:25:36.750 * 5 changes in 30 seconds. Saving...
598:S 19 Apr 2026 16:25:36.751 * Background saving started by pid 646
646:C 19 Apr 2026 16:25:36.761 * DB saved on disk
646:C 19 Apr 2026 16:25:36.762 * Fork CoW for RDB: current 0 MB, peak 0 MB, average 0 MB
598:S 19 Apr 2026 16:25:36.859 * Background saving terminated with success
598:S 19 Apr 2026 17:04:07.693 # Connection with master lost.
598:S 19 Apr 2026 17:04:07.693 * Caching the disconnected master state.
598:S 19 Apr 2026 17:04:07.693 * Reconnecting to MASTER 127.0.0.1:6379
598:S 19 Apr 2026 17:04:07.694 * MASTER <-> REPLICA sync started
598:S 19 Apr 2026 17:04:07.694 # Error condition on socket for SYNC: Connection refused
598:S 19 Apr 2026 17:04:08.015 * Connecting to MASTER 127.0.0.1:6379
598:S 19 Apr 2026 17:04:08.015 * MASTER <-> REPLICA sync started
598:S 19 Apr 2026 17:04:08.016 # Error condition on socket for SYNC: Connection refused
```

Відновимо primary

Після цього основний сервер було запущено повторно тією ж командою redis-server ./primary.conf із того самого каталогу. У журналі запуску було зафіксовано послідовне виконання процедури відновлення: зчитування базового знімка (Reading RDB base file on AOF loading, Loading RDB produced by version 7.0.15, Done loading RDB), завантаження інкрементного файлу журналу (DB loaded from append only file in 0.XXX seconds) та завершальне повідомлення Ready to accept connections tcp. Повернення сервера у робочий стан автоматично відновило й реплікаційний канал — у журналі replica з'явився рядок MASTER <-> REPLICA sync: Finished with success, що свідчило про успішну часткову ресинхронізацію (partial resync) без повторного повного перенесення знімка бази.





Перевірка, що дані справді відновились

Для підтвердження повноти відновлення даних було виконано серію перевіркових запитів через redis-cli. Команди GET persist:test1 та GET persist:test2 повернули значення "hello" та "world" відповідно — ті самі, що були записані до перезапуску у режимі appendfsync always. Команди GET foo, GET student, GET lab повернули значення "bar", "Nikita", "ready", які також були записані раніше і відтворились через AOF. Команда SCAN 0 вивела повний перелік ключів у базі, куди ввійшли всі записані за час роботи примари ключі, що підтвердило коректне збереження стану бази на диску і повноту відновлення після перезапуску сервера. Таким чином було експериментально доведено, що увімкнений режим appendfsync always разом із механізмом AOF забезпечує гарантію Durability — жодна операція запису, на яку клієнт отримав

відповідь OK, не втрачається при аварійному вимиканні та коректно відновлюється при повторному старті сервера.

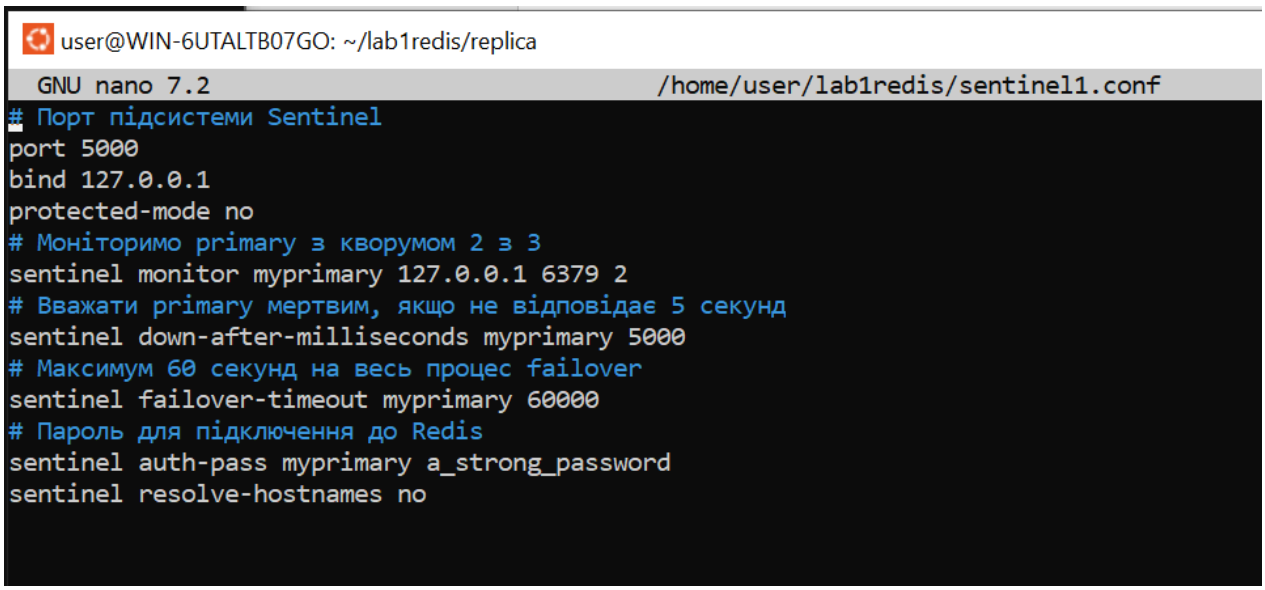
```
user@WIN-6UTALTB07GO:~/lab1redis/replica$ redis-cli -p 6379 -a a_strong_password --no-auth-warning
127.0.0.1:6379> GET persist:test1
"hello"
127.0.0.1:6379> GET persist:test2
"world"
127.0.0.1:6379> GET foo
"bar"
127.0.0.1:6379> GET student
"Nikita"
127.0.0.1:6379> GET lab
"ready"
127.0.0.1:6379> SCAN 0
1) "0"
2) 1) "student"
   2) "foo"
   3) "persist:test2"
   4) "lab"
   5) "persist:test1"
127.0.0.1:6379> EXIT
```

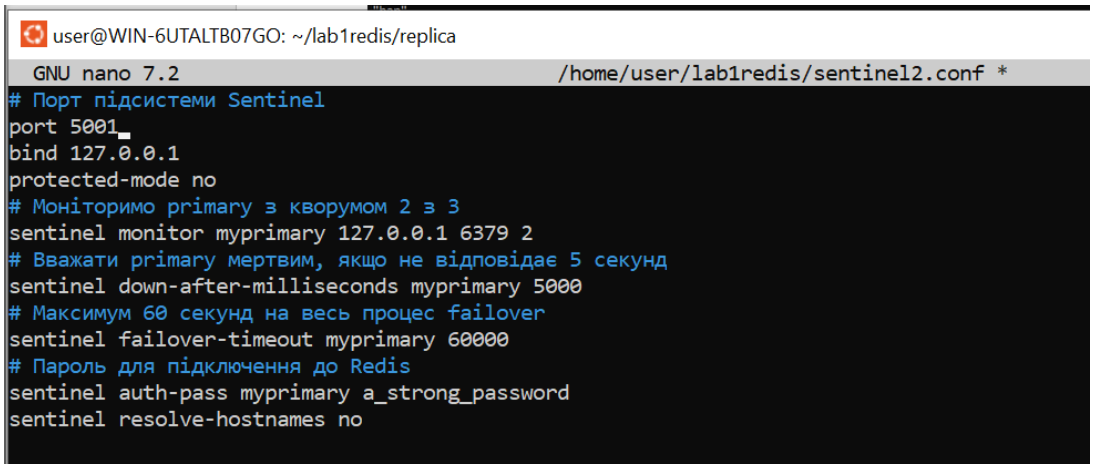
10) Створення та запуск трьох Sentinel

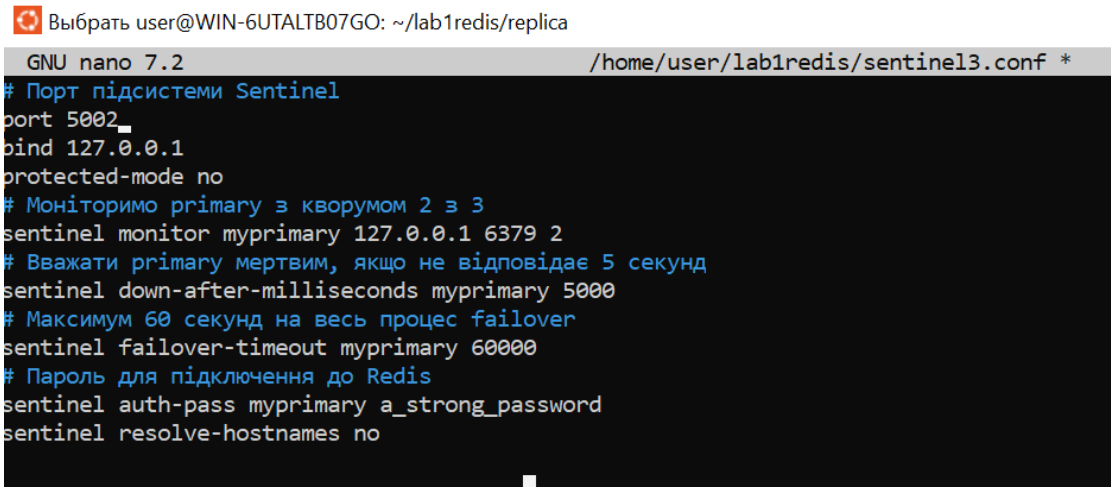
Для реалізації автономного відновлення (автоматичного failover) при аварійному вимиканні основного вузла було розгорнуто підсистему Redis Sentinel у складі трьох екземплярів, що забезпечує прийняття рішень кворумом і виключає split-brain при збоях мережі. У каталозі ~/lab1redis було створено три конфігураційні файли — sentinel1.conf, sentinel2.conf та sentinel3.conf — які відрізняються лише номером порту (відповідно 5000, 5001 та 5002). Усі інші параметри однакові у всіх файлах.

Ключова директива sentinel monitor myprimary 127.0.0.1 6379 2 задає ім'я контрольованого master-вузла (myprimary), його адресу та порт, а також кворум — мінімальну кількість Sentinel, які мають підтвердити недоступність master для ініціювання процесу failover. За умовою лабораторної роботи встановлено кворум 2 з 3. Директива sentinel down-after-milliseconds myprimary 5000 задає час, протягом якого Sentinel очікує відповіді від master; якщо master не відповідає понад 5 секунд, Sentinel позначає його як SDOWN (subjectively down) і повідомляє про це іншим Sentinel. Директива sentinel failover-timeout myprimary 60000 обмежує час виконання всього процесу делегування новим master до 60 секунд. Пароль для підключення Sentinel до захищених паролем Redis-вузлів задано директивою sentinel auth-pass myprimary a_strong_password, оскільки обидва наші екземпляри Redis працюють з увімкненою директивою requirepass.

```
user@WIN-6UTALTB07GO:~/lab1redis/replica$ nano ~/lab1redis/sentinel1.conf
user@WIN-6UTALTB07GO:~/lab1redis/replica$ nano ~/lab1redis/sentinel2.conf
user@WIN-6UTALTB07GO:~/lab1redis/replica$ nano ~/lab1redis/sentinel3.conf
```

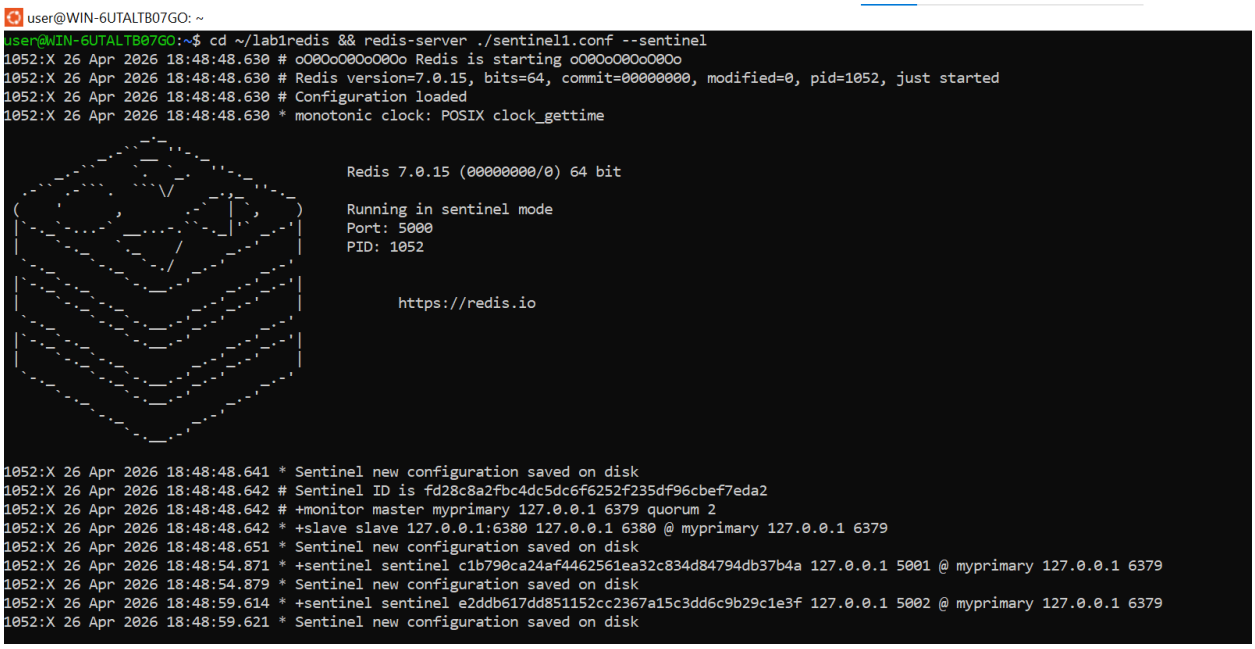


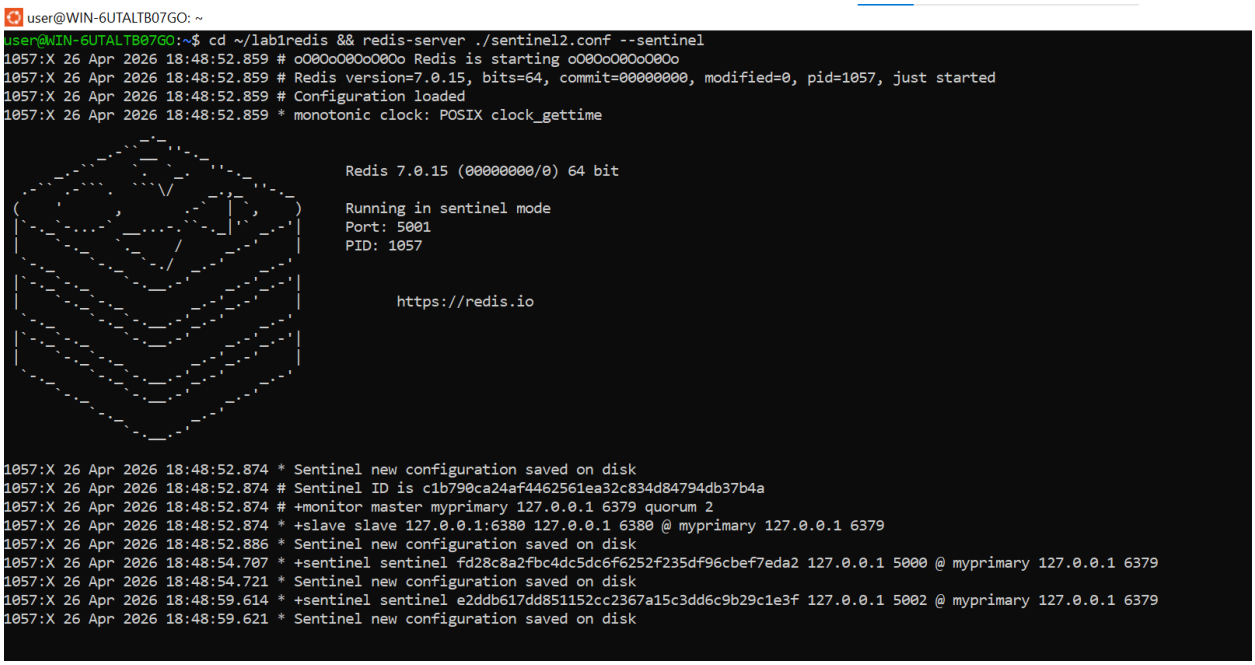


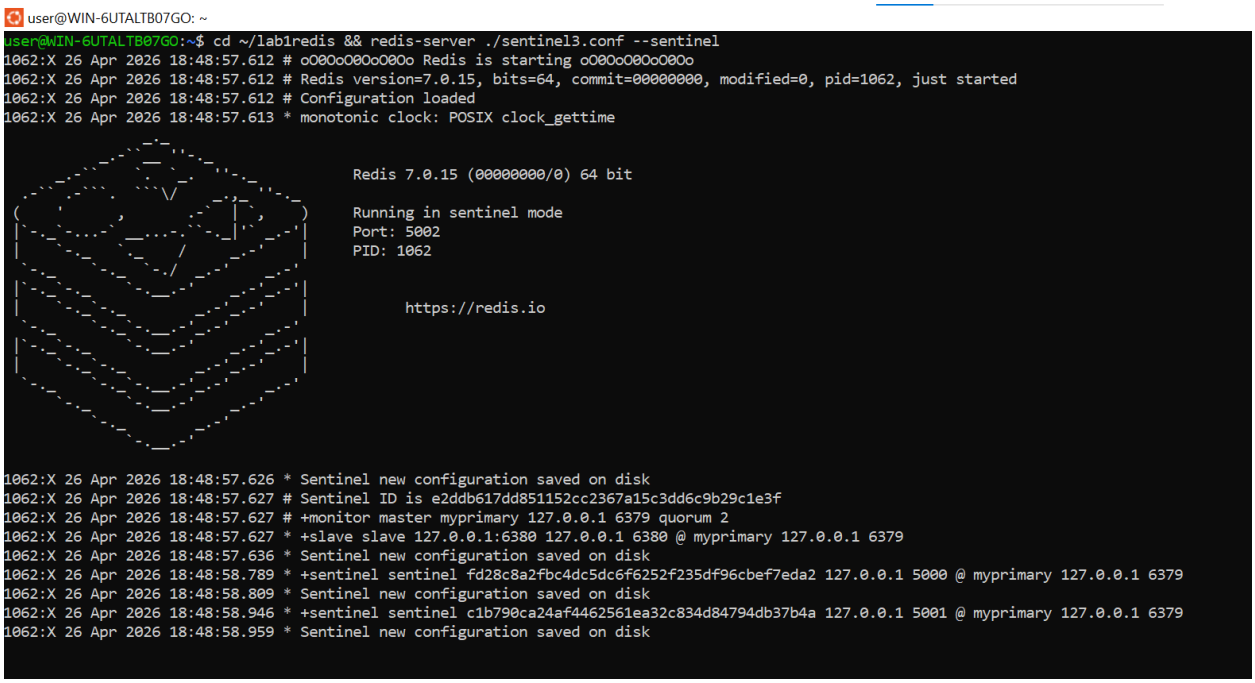


Конфігураційні файли sentinel2.conf та sentinel3.conf мають аналогічну структуру, у них змінено лише значення директиви port на 5001 та 5002 відповідно. Усі три екземпляри Sentinel було запущено у трьох окремих терміналах командою redis-server ~/lab1redis/sentinelN.conf --sentinel, де N — номер конфігураційного файлу. У журналі кожного Sentinel було зафіксовано

рядок +monitor master myprimary 127.0.0.1 6379 quorum 2, що підтвердило успішний старт моніторингу. Після запуску другого і третього Sentinel у журналах з'явились також записи +sentinel sentinel <id> 127.0.0.1 <port>, які свідчать про те, що Sentinel самостійно виявили одне одного через спеціальний Pub/Sub канал Redis __sentinel__:hello і сформували повноз'язану підсистему моніторингу.







Коректність роботи підсистеми було перевірено низкою команд через redis-cli на порту першого Sentinel. Команда SENTINEL master myprimary повернула детальну інформацію про контрольований master: адресу 127.0.0.1, порт 6379, прапорець master, кількість підключених slave-вузлів num-slaves: 1 та кількість інших Sentinel у підсистемі num-other-sentinels: 2. Команда SENTINEL sentinels myprimary повернула перелік з двох інших Sentinel на портах 5001 та 5002. Команда SENTINEL get-master-addr-by-name myprimary повернула пару 127.0.0.1 та 6379, тобто у поточний момент master — це наш основний сервер primary. Додатково командами INFO replication було перевірено, що вузол на порту 6379 має роль master із одним підключеним підлеглим (connected_slaves: 1), а вузол на порту 6380 — роль slave із активним реплікаційним каналом (master_link_status: up). Таким чином, початковий стан системи перед тестуванням failover повністю коректний.

11) Практична частина

Для виконання практичної частини лабораторної роботи було використа но програмне забезпечення на мові Python, яке взаємодіє з Redis через офіцій ний клієнтський модуль redis. Скрипт lab1.py був розміщений у каталозі ~/ lab1redis/code/ і запускався після того, як було піднято всі компоненти систем и: основний вузол (porty 6379), дублювальний вузол (порт 6380) та три екзем пляри Redis Sentinel (порти 5000, 5001, 5002).

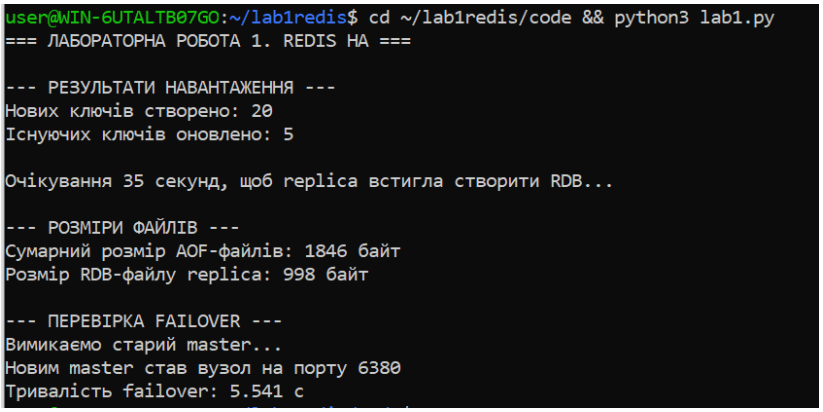
Скрипт реалізує такі функції:

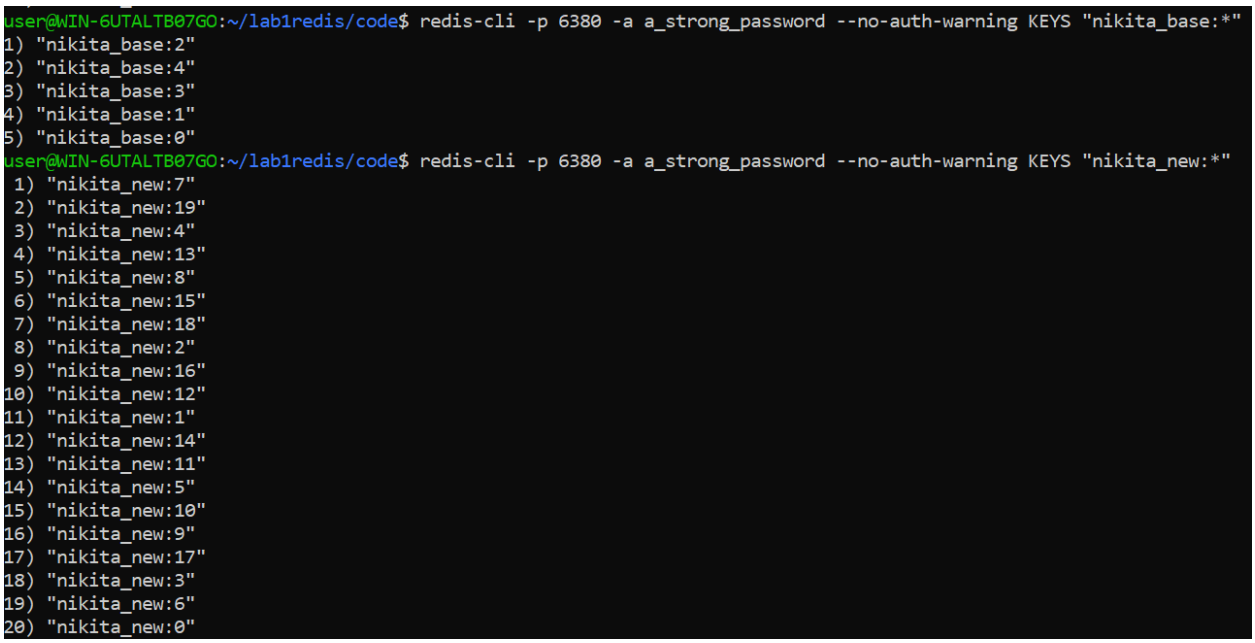
- generate_random_text — генерує випадковий рядок із літер та цифр (за замовчуванням 14 символів) для значень ключів;

- redis_client - створює клієнтське підключення до Redis-сервера за заданим портом із підтримкою автентифікації через requirepass;
- perform_probabilistic_load - виконує ймовірнісне навантаження на основний вузол: спочатку створює 5 базових ключів nikita_base: 0 … 4, після чого протягом до 60 секунд із паузами 1.2–2.8 с дозаписує 20 нових ключів nikita_new: * та оновлює 5 існуючих базових із ймовірністю 0.7 створення нового або оновлення існуючого;
- calculate_aof_size - обчислює сумарний розмір усіх файлів AOF у каталозі основного вузла, ураховуючи структуру appendonlydir із базовим base.rdb та інкрементним incr.aof файлами Redis 7;
- calculate_rdb_size - обчислює розмір RDB-файлу my_backup_file.rdb у каталозі дублювального вузла;
- stop_old_master - програмно вимикає основний вузол командою redis-cli -p 6379 -a a_strong_password shutdown nosave, що моделює аварійне завершення primary;
- wait_until_replica_becomes_master - у циклі через інтервали 0.5 с опитує дублювальний вузол командою INFO replication і фіксує момент, коли його роль змінюється з slave на master; різниця між моментом виклику функції та моментом зміни ролі і є тривалістю автоматичного делегування.

Підключення до серверів виконується через локальну адресу 127.0.0.1 із паролем a_strong_password та параметром decode_responses=True, що дозволяє отримувати рядкові значення без додаткового декодування

Результат роботи коду:





Висновок.

У ході виконання лабораторної роботи було досліджено механізми забезпечення високої доступності (High Availability) у системі Redis. Було налаштовано основний вузол primary з увімкненим журналюванням AOF, дублювальний вузол replica з RDB-файлом резервного збереження та правилом save 30 5, а також підсистему з трьох Redis Sentinel із кворумом 2 з 3, які забезпечують автономне делегування у випадку відмови основного сервера. Коректність реплікації було підтверджено командою MONITOR, надійність персистентності — перезапуском primary з успішним відновленням тестових ключів persist:test1 і persist:test2.

У практичній частині за допомогою Python-API було реалізовано ймовірнісне навантаження: протягом 60 секунд автоматично створено 20 нових ключів і оновлено 5 існуючих. Програмно обчислено розмір AOF-файлів 1846 байт і розмір RDB-файлу 998 байт. Після штучного вимикання основного вузла Redis Sentinel автоматично виконав делегування нового master, тривалість якого, виміряна програмно, склала 5.541 секунди. Жоден з 25 ключів не був втрачений під час перемикання, що підтвердило коректну роботу механізму реплікації.

Отже, Redis забезпечує одночасно надійне збереження даних (через комбінацію AOF та RDB) та автоматичну відмовостійкість сервісу (через Sentinel), що робить його придатним для побудови високодоступних систем з відносно низькою тривалістю переключення (одиниці секунд) і без втрати даних

Додаток А

```python
import os
import random
import string
import time
import subprocess
import redis

HOST = "127.0.0.1"
MASTER_PORT = 6379
REPLICA_PORT = 6380
REDIS_PASSWORD = "a_strong_password"

LAB_ROOT = os.path.expanduser("~/lab1redis")
MASTER_DIR = os.path.join(LAB_ROOT, "primary")
REPLICA_DIR = os.path.join(LAB_ROOT, "replica")
REPLICA_RDB_FILE = os.path.join(REPLICA_DIR, "my_backup_file.rdb")


def generate_random_text(length: int = 14) -> str:
    chars = string.ascii_letters + string.digits
    return "".join(random.choices(chars, k=length))


def redis_client(port: int) -> redis.Redis:
    return redis.Redis(
        host=HOST,
        port=port,
        password=REDIS_PASSWORD,
        decode_responses=True
    )


def calculate_aof_size(directory: str) -> int:
    total_size = 0

    for root, _, files in os.walk(directory):
        for file_name in files:
            full_path = os.path.join(root, file_name)
            lower_name = file_name.lower()

            if lower_name.endswith(".aof") or "appendonly" in lower_name:
                if os.path.isfile(full_path):
                    total_size += os.path.getsize(full_path)

    return total_size


def calculate_rdb_size(file_path: str) -> int:
    if os.path.exists(file_path) and os.path.isfile(file_path):
        return os.path.getsize(file_path)
    return 0
```

```
def wait_until_replica_becomes_master(timeout_seconds: int = 90) -> float |
None:
    start_time = time.time()

    while time.time() - start_time < timeout_seconds:
        try:
            replica = redis_client(REPLICA_PORT)
            replication_info = replica.info("replication")

            if replication_info.get("role") == "master":
                return time.time() - start_time
        except Exception:
            pass

        time.sleep(0.5)

    return None


def stop_old_master() -> None:
    subprocess.run(
        ["redis-cli", "-p", str(MASTER_PORT), "-a", REDIS_PASSWORD,
"shutdown", "nosave"],
        stdout=subprocess.DEVNULL,
        stderr=subprocess.DEVNULL,
        check=False
    )


def perform_probabilistic_load() -> tuple[int, int]:
    master = redis_client(MASTER_PORT)

    existing_keys = []
    created_count = 0
    updated_count = 0

    for i in range(5):
        key_name = f"nikita_base:{i}"
        master.set(key_name, generate_random_text())
        existing_keys.append(key_name)

    start_time = time.time()

    while time.time() - start_time < 60:
        create_new = False

        if created_count < 20 and updated_count >= 5:
            create_new = True
        elif created_count < 20 and updated_count < 5:
            create_new = random.random() < 0.7

        if create_new:
            key_name = f"nikita_new:{created_count}"
            value = generate_random_text()
            master.set(key_name, value)
            created_count += 1
        elif updated_count < 5 and existing_keys:
            key_name = random.choice(existing_keys)
            value = generate_random_text()
            master.set(key_name, value)
            updated_count += 1

        if created_count >= 20 and updated_count >= 5:
```

```
            break

        time.sleep(random.uniform(1.2, 2.8))

    return created_count, updated_count




def main():
    print("=== ЛАБОРАТОРНА РОБОТА 1. REDIS HA ===")

    try:
        created, updated = perform_probabilistic_load()
    except Exception as error:
        print(f"Помилка під час запису в master: {error}")
        return

    print("\n--- РЕЗУЛЬТАТИ НАВАНТАЖЕННЯ ---")
    print(f"Нових ключів створено: {created}")
    print(f"Існуючих ключів оновлено: {updated}")

    print("\nОчікування 35 секунд, щоб replica встигла створити RDB...")
    time.sleep(35)

    aof_bytes = calculate_aof_size(MASTER_DIR)
    rdb_bytes = calculate_rdb_size(REPLICA_RDB_FILE)

    print("\n--- РОЗМІРИ ФАЙЛІВ ---")
    print(f"Сумарний розмір AOF-файлів: {aof_bytes} байт")
    print(f"Розмір RDB-файлу replica: {rdb_bytes} байт")

    print("\n--- ПЕРЕВІРКА FAILOVER ---")
    print("Вимикаємо старий master...")

    stop_old_master()

    failover_duration = wait_until_replica_becomes_master()

    if failover_duration is not None:
        print(f"Новим master став вузол на порту {REPLICA_PORT}")
        print(f"Тривалість failover: {failover_duration:.3f} с")
    else:
        print("Не вдалося зафіксувати failover протягом 90 секунд")


if __name__ == "__main__":
    main()
```